

Git, Code Review, and CI That Protect Changes

Version control records decisions; code review checks risk and clarity; CI verifies repeatable contracts. Use all three together. A green pipeline is not proof of correctness, and a large commit message cannot compensate for an unreviewable change that mixes unrelated behavior.

For: Developers who can commit code but want safer collaboration, reviewable changes, meaningful automated checks, and an understandable recovery workflow.

Suggested pace: 3 weeks on a small personal repository | Beginner to intermediate

What you will be able to do

- Inspect and preserve local and remote changes before editing or publishing.
- Create small coherent commits and review descriptions grounded in behavior.
- Build CI checks that test meaningful risks and fail clearly.
- Recover through safe reversible operations instead of destructive guessing.

Build the right mental model

Understand the three states

Working files, the staging area, and committed history are different. Inspect status and diffs before committing. Fetching updates remote-tracking information; it does not by itself integrate another person's changes into your working branch.

Review the behavioral contract

Explain what changed, why, and how it was checked. A reviewer needs the relevant assumptions, failure cases, and compatibility risks. Keep refactoring separate from behavior changes when that improves reviewability rather than obscuring causality.

CI is executable agreement

Run formatting or linting, focused tests, and build checks appropriate to the project. Use reproducible dependency versions and avoid secrets in logs. Tests should fail on a real broken condition, not merely execute code without asserting behavior.

Study with evidence.

Practice on a repository you own with a small feature and regression test. Do not rewrite shared history without agreement. When a worktree contains changes you did not make, preserve them and understand their relationship to your task.

Week 1: State and history

Practice status, diff, staging, commit, log, branches, fetch, and merge using disposable examples. Introduce a conflicting edit and resolve it by understanding both changes rather than selecting one side blindly.

Deliverable and check: A small history whose commits each have one purpose and whose conflict resolution preserves both intended behaviors.

Week 2: Review and regression tests

Make a narrow bug fix with a test that fails before the fix and passes afterward. Write a review description covering cause, behavior, validation, and residual risk. Review another small change for edge cases.

Deliverable and check: A reviewable patch and a regression test that demonstrates the failure mechanism.

Week 3: Automate and recover

Add CI for the agreed checks and test it on a deliberately failing branch. Practice reverting a published change with a new commit. Document release and rollback steps and avoid relying on a local-only environment.

Deliverable and check: A pipeline that catches the known regression and a recovery exercise that preserves shared history.

Ongoing: Keep feedback useful

Remove flaky assumptions, keep test fixtures deterministic, and record why a check exists. Review dependency updates and permissions. Prefer a small set of trustworthy gates over many ignored red builds.

Deliverable and check: A maintained checklist linking each required gate to a meaningful project risk.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Fix a bug while preserving another contributor's work

Your local repository is clean, but the remote branch has gained changes since your last fetch. You need to publish a small fix without discarding a newer feature or forcing an overwrite.

A worked approach

1. Fetch and inspect the remote difference before editing or integrating. Identify whether the newer changes affect the same contract. A clean status relative to stale tracking information does not prove the remote is unchanged.
2. Create a focused fix and a regression test. Integrate remote work using the team's agreed merge or rebase workflow, resolving conflicts from behavior rather than file ownership. Rerun the focused check after integration.
3. Push without force when possible and inspect the actual CI or deployment result. Verify the intended version, then report what was checked and any remaining uncertainty instead of equating a successful push with a successful release.

Expected result

The fix and the other contributor's feature remain intact, the regression is covered, and publication is verified. The process protects both history and behavior.

Verify the result

- Was the remote state refreshed before publication?
- Does the conflict resolution preserve both intended changes?
- Did the deployment verify the exact commit rather than an older successful run?

Extend the exercise

Simulate a deployment regression and use a revert commit or the documented rollback mechanism. Explain why deleting local files or resetting shared history is not a substitute for a controlled recovery.

Make the learning stick.

Is a green CI run enough to approve a change?

No. It verifies only the checks that ran under their assumptions. Review design, edge cases, security, and compatibility, and state untested risks. A test suite can be green while missing the changed contract entirely.

Should every change be one commit?

A commit should represent a coherent step that can be understood and, ideally, tested. Several focused commits may be better than one enormous change; many arbitrary fragments may be worse. Follow the team's review conventions.

When is force-pushing appropriate?

Only within an agreed workflow and with care for shared work. It rewrites remote history and can disrupt others. Prefer non-destructive integration and obtain approval before rewriting a shared branch.

Avoid these traps

- Publishing against stale remote-tracking information.
- Resolving conflicts by discarding unfamiliar changes.
- Treating lint or coverage percentage as proof of behavioral correctness.

Ready for the next step?

- Working, staged, committed, and remote states are understood.
- A regression test demonstrates the bug being fixed.
- Review descriptions identify risks and verification.
- Release and rollback steps have been exercised safely.

Reliable next reads

- [Pro Git book](#)
- [GitHub Actions documentation](#)
- [Google engineering practices](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.